LAPORAN TUGAS KECIL 3

IF2211 Strategi Algoritma

Ice Sliding Puzzle Solver

Disusun oleh:

Stanislaus Ardy Bramantyo

18223057

Fudhal Fayyadh

18223121

Daftar ISI

Bab I — Pendahuluan **5**
1.1 Deskripsi Masalah 5
1.2 Tujuan 6
Bab II — Algoritma Pathfinding **7**
2.1 Uniform Cost Search (UCS) 7
2.1.1 Deskripsi Algoritma 7
2.1.2 Definisi f(n) dan g(n) 7
2.1.3 Langkah-langkah Algoritma 7
2.1.4 Implementasi 8
2.2 Greedy Best-First Search (GBFS) 8
2.2.1 Deskripsi Algoritma 8
2.2.2 Definisi f(n) dan g(n) 8
2.2.3 Langkah-langkah Algoritma 9
2.2.4 Implementasi 9
2.3 A* (A-Star) 9
2.3.1 Deskripsi Algoritma 10
2.3.2 Definisi f(n) dan g(n) 10
2.3.3 Langkah-langkah Algoritma 10
2.3.4 Implementasi 11
2.4 Fungsi Heuristik 11
2.4.1 H1 — Manhattan Distance 11
2.4.2 H2 — Euclidean Distance 12
2.4.3 H3 — Chebyshev Distance 12
Bab III — Analisis Algoritma **13**
3.1 Admissibility Heuristik pada A* 13
3.1.1 Analisis Admissibility H1 (Manhattan Distance) 13
3.1.2 Analisis Admissibility H2 (Euclidean Distance) 14
3.1.3 Analisis Admissibility H3 (Chebyshev Distance) 14
3.2 Perbandingan Teoritis Algoritma 15
3.3 Analisis Kompleksitas pada Ice Sliding Puzzle 15
Bab IV — Implementasi Program **16**
4.1 Bahasa Pemrograman dan Struktur Program 16
4.2 Representasi State 17
4.3 Mekanisme Sliding 18
4.4 Perhitungan Cost 18
4.5 Format Input 19
4.6 Format Output 19
Bab V — Hasil Percobaan **20**
5.1 Contoh Test Case 1 20
5.2 Contoh Test Case 2 22
5.3 Contoh Test Case 3 25
5.4 Contoh Test Case 4 28
5.5 Contoh Test Case 5 31
5.6 Analisis Hasil Percobaan 34
5.6.1 Perbandingan Cost Solusi 34

5.6.2 Perbandingan Jumlah Iterasi 35
5.6.3 Kesimpulan Analisis 36
Bab VI — Source Code **37**
6.1 Repository 37

Daftar Gambar

Gambar 5.1.1 input test case 1 20
Gambar 5..1.2 Output UCS test case 1 21
Gambar 5.1.3 Output GBFS H1(test case 1) 21
Gambar 5.1.4 Output GBFS H2(test case 1) 21
Gambar 5.1.5 Output GBFS H3(test case 1) 21
Gambar 5.1.6 Output A* H1(test case 1) 22
Gambar 5.1.7 Output A* H1(test case 1) 22
Gambar 5.1.8 Output A* H1(test case 1) 22
Gambar 5.2.1 input test case 2 23
Gambar 5.2.2 Output UCS test case 2 24
Gambar 5.2.3 Output GBFS H1(test case 2) 24
Gambar 5.2.4 Output GBFS H2(test case 2) 24
Gambar 5.2.5 Output GBFS H3(test case 2) 24
Gambar 5.2.6 Output A* H1(test case 2) 25
Gambar 5.2.7 Output A* H2(test case 2) 25
Gambar 5.2.8 Output A* H3(test case 2) 25
Gambar 5.3.1 input test case 3 26
Gambar 5.3.2 Output UCS test case 3 27
Gambar 5.3.3 Output GBFS H1(test case 3) 27
Gambar 5.3.4 Output GBFS H2(test case 3) 27
Gambar 5.3.5 Output GBFS H3(test case 3) 27
Gambar 5.3.6 Output A* H1(test case 3) 28
Gambar 5.3.7 Output A* H2(test case 3) 28
Gambar 5.3.8 Output A* H3(test case 3) 28
Gambar 5.4.1 input test case 4 29
Gambar 5.4.2 Output UCS test case 4 30
Gambar 5.4.3 Output GBFS H1(test case 4) 30
Gambar 5.4.4 Output GBFS H2(test case 4) 30
Gambar 5.4.5 Output GBFS H3(test case 4) 30
Gambar 5.4.6 Output A* H1(test case 4) 31
Gambar 5.4.7 Output A* H2(test case 4) 31
Gambar 5.4.8 Output A* H3(test case 4) 31
Gambar 5.5.1 input test case 5 32
Gambar 5.5.2 Output UCS test case 5 33
Gambar 5.5.3 Output GBFS H1(test case 5) 34
Gambar 5.5.4 Output GBFS H2(test case 5) 34
Gambar 5.5.5 Output GBFS H3(test case 5) 34
Gambar 5.5.6 Output A* H1(test case 5) 34

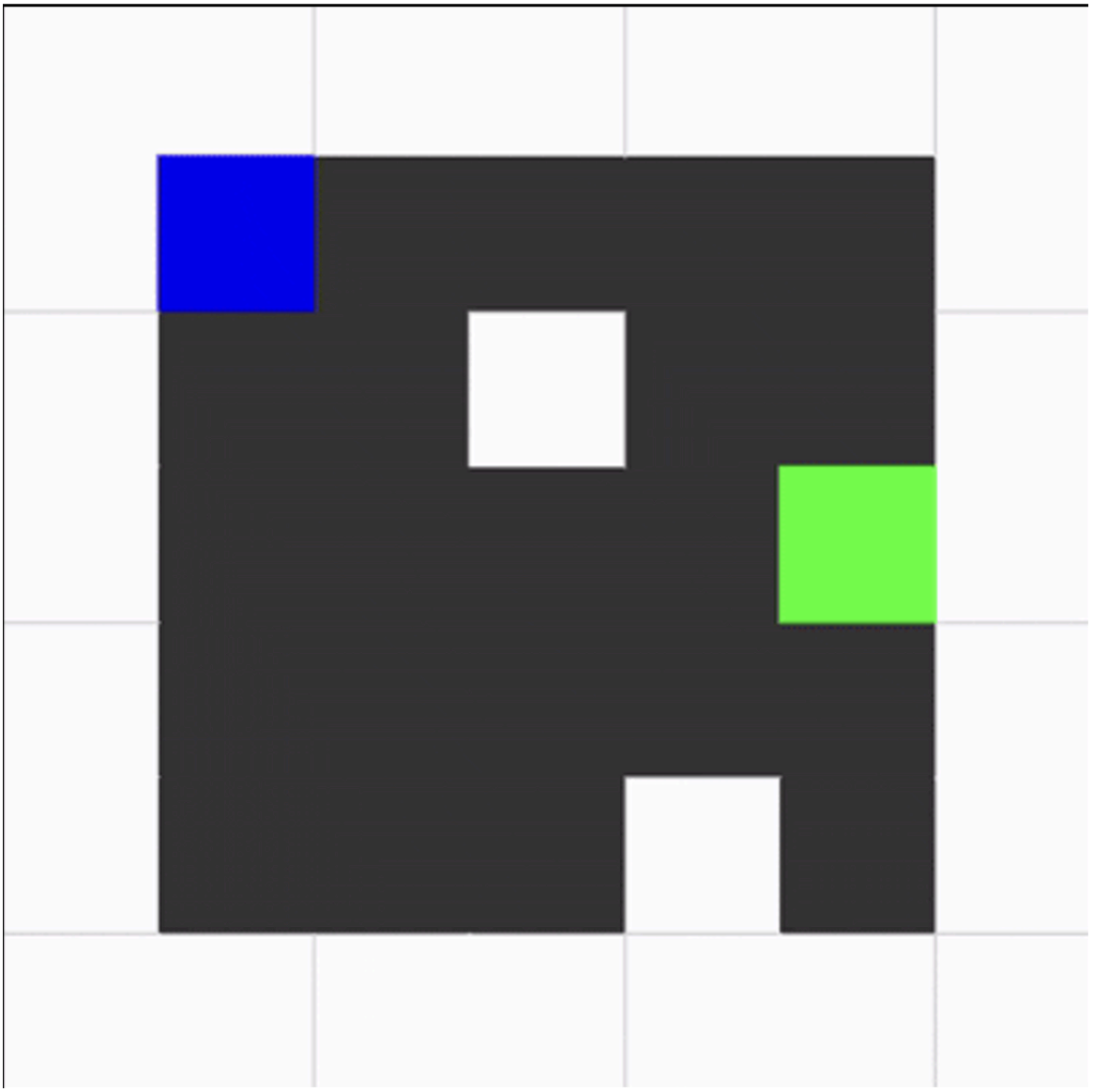
Gambar 5.5.7 Output A* H2(test case 5) 34
Gambar 5.5.8 Output A* H3(test case 5) 34
Tabel 5.6.1.1 perbandingan cost solusi 35
Tabel 5.6.2.1 perbandingan jumlah iterasi solusi 35

Daftar Tabel

Tabel 5.6.1.1 perbandingan cost solusi 35
Tabel 5.6.2.1 perbandingan jumlah iterasi solusi 35

Bab I — Pendahuluan

1.1 Deskripsi Masalah



Ice Sliding Puzzle adalah permainan logika di mana pemain harus menggerakkan karakter dari titik awal menuju titik keluar di atas permukaan es yang licin. Pin (dalam visualisasi warna biru) hanya dapat bergerak secara horizontal atau vertikal, namun karena permukaan licin, karakter tidak akan berhenti bergerak sampai menabrak dinding atau rintangan (dalam visualisasi warna putih).

1.2 Tujuan

1. Mengimplementasikan algoritma pathfinding Uniform Cost Search (UCS), Greedy Best-First Search (GBFS), dan A* (A-Star) dalam bahasa pemrograman Go untuk menyelesaikan permasalahan Ice Sliding Puzzle.
2. Memahami perbedaan mendasar antara ketiga algoritma tersebut, khususnya dalam hal cara menentukan prioritas ekspansi node, jaminan optimalitas solusi, dan efisiensi pencarian.
3. Mengimplementasikan tiga fungsi heuristik (Manhattan Distance, Euclidean Distance, dan Chebyshev Distance) yang digunakan oleh GBFS dan A*, serta menganalisis sifat admissibility dari masing-masing heuristik.
4. Memvisualisasikan proses pencarian solusi secara step-by-step melalui antarmuka CLI maupun GUI, sehingga alur pergerakan pin dari posisi awal menuju posisi tujuan dapat dipahami dengan jelas.
5. Membandingkan kinerja ketiga algoritma berdasarkan cost solusi yang dihasilkan, jumlah iterasi yang dilakukan, dan waktu eksekusi program pada berbagai konfigurasi peta yang berbeda.
6. Menganalisis pengaruh pemilihan algoritma dan heuristik terhadap kualitas dan efisiensi solusi yang ditemukan dalam konteks permasalahan Ice Sliding Puzzle.

Bab II — Algoritma Pathfinding

2.1 Uniform Cost Search (UCS)

2.1.1 Deskripsi Algoritma

Uniform Cost Search (UCS) adalah algoritma pencarian yang selalu memperluas node dengan cost aktual terendah dari state awal. Algoritma ini tidak menggunakan informasi heuristik apapun, sehingga termasuk dalam kategori uninformed search. UCS menjamin solusi optimal selama semua cost bernilai positif, karena ia selalu memproses jalur dengan akumulasi biaya terkecil terlebih dahulu.

Pada implementasi Ice Sliding Puzzle ini, UCS menggunakan priority queue (min-heap) yang mengurutkan node berdasarkan nilai g(n), yaitu total cost aktual dari state awal hingga state n. Setiap kali sebuah node diekspansi, algoritma mensimulasikan empat kemungkinan gerakan sliding (atas, bawah, kiri, kanan) dan menghitung cost masing-masing gerakan berdasarkan jumlah cost tile yang dilewati. Node successor yang valid kemudian dimasukkan ke dalam priority queue dengan cost yang telah diperbarui. Proses ini berlanjut hingga state tujuan ditemukan atau priority queue kosong (tidak ada solusi).

Algoritma juga menyimpan visitedList berupa map dari State ke cost terbaik yang pernah ditemukan untuk state tersebut. Jika suatu state sudah pernah dikunjungi dengan cost yang lebih rendah atau sama, node tersebut akan dilewati agar tidak terjadi ekspansi yang redundan.

2.1.2 Definisi f(n) dan g(n)

Pada UCS terdapat komponen, yaitu:

- **g(n)**: total cost aktual yang diakumulasikan dari state awal hingga state n. Nilai ini dihitung sebagai penjumlahan seluruh cost tile yang dilewati selama setiap gerakan sliding sepanjang jalur dari state awal ke state n. Cost tile awal tidak dihitung, sedangkan cost tile tujuan setiap gerakan tetap dihitung.
- **f(n)**: pada UCS, f(n) = g(n). Tidak ada komponen heuristik, sehingga prioritas ekspansi node sepenuhnya ditentukan oleh cost aktual yang telah ditempuh.

2.1.3 Langkah-langkah Algoritma

Langkah Langkah algoritma UCS adalah sebagai berikut:

1. **Inisialisasi**: Buat node awal dari InitialState() dengan g(start) = 0 dan masukkan ke dalam priority queue berdasarkan nilai f(start) = g(start) = 0.
2. **Iterasi utama**: Selama priority queue tidak kosong, ambil node dengan nilai f(n) = g(n) terkecil.
3. **Pengecekan visited**: Jika state dari node yang diambil sudah pernah dikunjungi dengan cost yang lebih rendah atau sama, lewati node ini dan lanjutkan ke iterasi berikutnya.
4. **Perbarui visited**: Catat state tersebut beserta cost aktualnya ke dalam visitedList.
5. **Pengecekan tujuan**: Periksa apakah state saat ini merupakan state tujuan (posisi pin berada di GoalRow, GoalCol, dan seluruh checkpoint telah dilalui sesuai urutan). Jika ya, kembalikan solusi beserta total cost, jumlah iterasi, dan waktu eksekusi.

6. **Ekspansi**: Untuk setiap arah gerakan yang mungkin (atas, bawah, kiri, kanan), jalankan fungsi Slide untuk mensimulasikan pergerakan pin hingga berhenti.
    a. Jika gerakan tidak valid (keluar peta tanpa dinding, melewati lava, atau melanggar urutan checkpoint), abaikan.
    b. Jika valid, hitung cost gerakan sebagai jumlah cost seluruh tile yang dilewati selama sliding.
    c. Hitung g(successor) = g(n) + cost gerakan.
7. **Pruning**: Jika state successor sudah pernah dikunjungi dengan cost lebih rendah atau sama, abaikan successor tersebut.
8. **Tambahkan ke antrian**: Buat node baru untuk successor dengan g(successor) dan f(successor) = g(successor), lalu masukkan ke priority queue.
9. **Terminasi gagal**: Jika priority queue kosong dan tujuan belum ditemukan, kembalikan hasil dengan Found = false, menandakan tidak ada solusi yang tersedia.

2.1.4 Implementasi

Pada implementasi, UCS menggunakan fungsi prioCount yang hanya mengembalikan nilai g(n):

```
func SolveUCS(board *Board) SolveResult {
    prioCount := func(g int, s State) int {
        return g
    }
    return solve(board, prioCount)
}
```

2.2 Greedy Best-First Search (GBFS)

2.2.1 Deskripsi Algoritma

Greedy Best-First Search (GBFS) merupakan algoritma pencarian yang pemilihan node berikutnya berdasarkan nilai heuristiknya saja(cost tidak dalam perhitungan). Algoritma ini termasuk dalam kategori informed search, yang dimana algoritma ini akan menggunakan informasi dari heuristic dalam pencarian solusi. GBFS ini bersifat serakah yang akan selalu memilih ark yang terlihat paling dekat tanpa memperhatikan cost yang akan digunakan.

Pada ice sliding puzzle ini, GBFS akan menggunakan priority queue yang akan mengurutkan node berdasarkan nilai heuristiknya (estimasi jarak dari state awal). Pada saat pemilihan gerakan algoritma ini akan mensimulasikan empat gerakan dan menghitung nilai heuristik setiap successornya untuk menentukan arah gerakannya. Karena GBFS tidak memperhatikan g(n) (cost gerakan), solusi ini tidak efektif untuk mencari gerakan dengan cost terendah, namun algoritma ini cocok untuk menentukan solusi dengan nilai iterasi paling sedikit.

2.2.2 Definisi f(n) dan g(n)

Pada GBFS terdapat komponen yang perlu dipahami yaitu:

- **g(n):** Total cost yang diakumulasikan pada algoritma ini. Walaupun algoritma ini tidak menggunakan total cost dalam pencarian node selanjutnya, namun total cost ini digunakan untuk pelaporan cost solusi yang digunakan

- **h(n):** estimasi nilai heuristik dari state awal ke goal state yang dihitung menggunakan salah satu dari tiga heuristik yang digunakan (Manhattan, Euclidean, atau Chebyshev).
- **f(n):** pada GBFS, f(n) = h(n). Prioritas ekspansi node sepenuhnya ditentukan oleh estimasi jarak ke tujuan, tanpa memperhitungkan cost yang sudah ditempuh.

2.2.3 Langkah-langkah Algoritma

Langkah Langkah algoritma GBFS adalah sebgai berikut:

1. **Inisialisasi**: Buat node awal dari InitialState() dengan g(start) = 0 dan masukkan ke dalam priority queue berdasarkan nilai f(start) = h(start), yaitu estimasi heuristik dari posisi awal ke tujuan.
2. **Iterasi utama**: Selama priority queue tidak kosong, ambil node dengan nilai f(n) = h(n) terkecil.
3. **Pengecekan visited**: Jika state dari node yang diambil sudah pernah dikunjungi dengan cost yang lebih rendah atau sama, lewati node ini dan lanjutkan ke iterasi berikutnya.
4. **Perbarui visited**: Catat state tersebut beserta cost aktualnya ke dalam visitedList.
5. **Pengecekan tujuan**: Periksa apakah state saat ini merupakan state tujuan (posisi pin berada di GoalRow, GoalCol, dan seluruh checkpoint telah dilalui sesuai urutan). Jika ya, kembalikan solusi beserta total cost, jumlah iterasi, dan waktu eksekusi.
6. **Ekspansi**: Untuk setiap arah gerakan yang mungkin (atas, bawah, kiri, kanan), jalankan fungsi Slide untuk mensimulasikan pergerakan pin hingga berhenti.
    a. Jika gerakan tidak valid (keluar peta tanpa dinding, melewati lava, atau melanggar urutan checkpoint), abaikan.
    b. Jika valid, hitung g(successor) = g(n) + cost gerakan untuk keperluan pelaporan.
    c. Hitung h(successor) menggunakan fungsi heuristik yang dipilih sebagai nilai prioritas.
7. **Pruning**: Jika state successor sudah pernah dikunjungi dengan cost lebih rendah atau sama, abaikan successor tersebut.
8. **Tambahkan ke antrian**: Buat node baru untuk successor dengan f(successor) = h(successor), lalu masukkan ke priority queue.
9. **Terminasi gagal**: Jika priority queue kosong dan tujuan belum ditemukan, kembalikan hasil dengan Found = false, menandakan tidak ada solusi yang tersedia.

2.2.4 Implementasi

Pada implementasi, GBFS menggunakan fungsi prioCount yang hanya mengembalikan nilai heuristik h(n), mengabaikan g(n):

```go
func SolveGBFS(board *Board, heuristik HeuristicFn) SolveResult {
    prioCount := func(g int, s State) int {
        return heuristik(s, board)
    }
    return solve(board, prioCount)
}
```

2.3 A* (A-Star)

2.3.1 Deskripsi Algoritma

A* adalah sebuah algoritma yang menggabungkan keunggulan UCS dan GBFS dengan cara menggunakan f(n) = g(n) + h(n)(penggabungan antara cost dan jarak tempuh). Algoritma ini termasuk dalam kategori informed search yang akan menjamin solusi optimal selama heuristik yang digunakan admissible.

Penggunaan algoritma A* pada ice sliding puzzle menggunakan priority queue yang mengurutkan node berdasarkan nilai cost dan jarak tempuh yang mengakibatkan A* tidak hanya mengejar node terdekat (seperti GBFS) namun juga memperhitungkan cost yang digunakan untuk mencapai node tujuan (seperti UCS). hal ini membuat A* dapat mencari solusi paling optimal dibandingkan UCS dan GBFS.

Seperti halnya UCS dan GBFS, A* juga menyimpan visitedList berupa map dari State ke cost terbaik yang pernah ditemukan. Jika suatu state sudah pernah dikunjungi dengan cost yang lebih rendah atau sama, node tersebut akan dilewati untuk menghindari ekspansi yang redundan.

2.3.2 Definisi f(n) dan g(n)

Pada algoritma A* terdapat tiga komponen yang digunakan dan saling berkerja sama:

- **g(n):** total cost yyang diakumulasikan dari awal state hingga state n(goal). Nilai ini dihitung sebagai penjumlahan dari semua cost tile yang dilewati.
- **h(n):** estimasi heuristik yang diakumulasikan dari state awal ke state tujuan. Niai ini dihitung menggunakan salah satu dari tiga fungsi heuristik yang tersedia (Manhattan, Euclidean, atau Chebyshev).
- **f(n):** Pada A* f(n) dihitung dengan cara pertambahan antara cost dan heuristiknya (g(n) + h(n)). Prioritas ekspansi node ditentukan perdasarkan nilai f(n) terkecil.

2.3.3 Langkah-langkah Algoritma

Langkah Langakah algoritma A* adalah sebagai berikut:

1. **Inisialisasi**: Buat node awal dari InitialState() dengan g(start) = 0 dan masukkan ke dalam priority queue berdasarkan nilai f(start) = g(start) + h(start) = 0 + h(start).
2. **Iterasi utama**: Selama priority queue tidak kosong, ambil node dengan nilai f(n) terkecil.
3. **Pengecekan visited**: Jika state dari node yang diambil sudah pernah dikunjungi dengan cost yang lebih rendah atau sama, lewati node ini dan lanjutkan ke iterasi berikutnya.
4. **Perbarui visited**: Catat state tersebut beserta cost aktual g(n) ke dalam visitedList.
5. **Pengecekan tujuan**: Periksa apakah state saat ini merupakan state tujuan (posisi pin berada di GoalRow, GoalCol, dan seluruh checkpoint telah dilalui sesuai urutan). Jika ya, kembalikan solusi beserta total cost, jumlah iterasi, dan waktu eksekusi.
6. **Ekspansi**: Untuk setiap arah gerakan yang mungkin (atas, bawah, kiri, kanan), jalankan fungsi Slide untuk mensimulasikan pergerakan pin hingga berhenti.
    a. Jika gerakan tidak valid (keluar peta tanpa dinding, melewati lava, atau melanggar urutan checkpoint), abaikan.
    b. Jika valid, hitung cost gerakan sebagai jumlah cost seluruh tile yang dilewati selama sliding.
    c. Hitung g(successor) = g(n) + cost gerakan.
    d. Hitung h(successor) menggunakan fungsi heuristik yang dipilih.
    e. Hitung f(successor) = g(successor) + h(successor).
7. **Pruning**: Jika state successor sudah pernah dikunjungi dengan cost lebih rendah atau sama, abaikan successor tersebut.

8. **Tambahkan ke antrian**: Buat node baru untuk successor dengan f(successor) = g(successor) + h(successor), lalu masukkan ke priority queue.
9. **Terminasi gagal**: Jika priority queue kosong dan tujuan belum ditemukan, kembalikan hasil dengan Found = false, menandakan tidak ada solusi yang tersedia.

2.3.4 Implementasi

Pada implementasi, A* menggunakan fungsi prioCount yang menjumlahkan g(n) dan h(n):

```
func SolveAStar(board *Board, heuristik HeuristicFn) SolveResult {
    prioCount := func(g int, s State) int {
        return g + heuristik (s, board)
    }
    return solve(board, prioCount)
}
```

2.4 Fungsi Heuristik

Program mengimplementasikan tiga fungsi heuristik yang dapat dipilih untuk GBFS dan A*. Ketiganya mengukur estimasi jarak dari posisi pin saat ini ke posisi tujuan (GoalRow, GoalCol).

2.4.1 H1 — Manhattan Distance

Manhattan Distance menghitung estimasi jarak dari posisi pin saat ini ke posisi tujuan dengan menjumlahkan selisih absolut baris dan kolom antara kedua posisi tersebut. Formula yang digunakan adalah:

```
func H1Manhattan(s State, b *Board) int {
    selisihBaris := s.Row - b.GoalRow
    if selisihBaris < 0 {
        selisihBaris = -selisihBaris
    }

    selisihKolom := s.Col - b.GoalCol
    if selisihKolom < 0 {
        selisihKolom = -selisihKolom
    }

    hasilAkhir := selisihBaris + selisihKolom
    return hasilAkhir
}
```

Heuristik ini cocok untuk pergerakan 4 arah (atas, bawah, kiri, kanan) karena merepresentasikan jumlah minimum perpindahan tile yang harus dilakukan secara horizontal dan vertikal untuk mencapai tujuan, tanpa memperhitungkan rintangan di antara keduanya. Pada Ice Sliding Puzzle, karena pin bergerak secara sliding melewati banyak tile sekaligus dalam satu gerakan, biaya aktual selalu lebih besar atau sama dengan nilai Manhattan Distance, sehingga heuristik ini bersifat admissible.

2.4.2 H2 — Euclidean Distance

Euclidean Distance menghitung estimasi jarak dari posisi pin saat ini ke posisi tujuan berdasarkan jarak lurus (garis lurus) antara kedua posisi, kemudian dibulatkan ke bawah (floor) menjadi integer. Formula yang digunakan adalah:

```go
func H2Euclidean(s State, b *Board) int {
    selisihBaris := s.Row - b.GoalRow
    selisihKolom := s.Col - b.GoalCol

    floatSB := float64(selisihBaris)
    floatSK := float64(selisihKolom)

    kuadratSB := floatSB * floatSB
    kuadratSK := floatSK * floatSK

    // membulatkan ke bawah
    total := kuadratSB+kuadratSK
    rootTotal := math.Sqrt(total)
    hasilAkhir := int (rootTotal)

    return hasilAkhir
}
```

Heuristik ini menghasilkan nilai yang selalu lebih kecil atau sama dengan Manhattan Distance karena jarak lurus selalu lebih pendek atau sama dengan jarak yang ditempuh melalui jalur horizontal dan vertikal. Oleh karena itu, jika Manhattan Distance sudah admissible, maka Euclidean Distance juga pasti admissible. Namun karena nilainya lebih kecil dari Manhattan, heuristik ini cenderung kurang informatif sehingga dapat menghasilkan lebih banyak ekspansi node dibandingkan H1 pada beberapa kasus.

2.4.3 H3 — Chebyshev Distance

Chebyshev Distance menghitung estimasi jarak dari posisi pin saat ini ke posisi tujuan dengan mengambil nilai maksimum antara selisih absolut baris dan selisih absolut kolom. Formula yang digunakan adalah:

```go
func H3Chebyshev(s State, b *Board) int {
    selisihBaris := s.Row - b.GoalRow
    if selisihBaris < 0 {
        selisihBaris = -selisihBaris
```

```
    }

    selisihKolom := s.Col - b.GoalCol
    if selisihKolom < 0 {
        selisihKolom = -selisihKolom
    }

    var hasilAkhir int

    if selisihBaris > selisihKolom {
        hasilAkhir = selisihBaris
    } else {
        hasilAkhir = selisihKolom
    }

    return hasilAkhir
}
```

Heuristik ini umumnya digunakan pada pergerakan 8 arah (termasuk diagonal), di mana satu langkah diagonal dapat mengurangi jarak baris dan kolom sekaligus. Pada Ice Sliding Puzzle yang hanya memiliki pergerakan 4 arah, Chebyshev Distance menghasilkan nilai yang selalu lebih kecil atau sama dengan Manhattan Distance karena max(|dr|, |dc|) ≤ |dr| + |dc| untuk semua nilai dr dan dc. Dengan demikian, heuristik ini juga bersifat admissible, meskipun nilainya lebih rendah dari H1 sehingga panduan yang diberikan ke algoritma pencarian menjadi kurang presisi dibandingkan Manhattan Distance.

Bab III — Analisis Algoritma

3.1 Admissibility Heuristik pada A*

Sebuah heuristik $h(n)$ dikatakan **admissible** jika untuk setiap simpul n pada ruang *search* berlaku:

$$h(n) \leq h'(n)$$

Dalam formula di atas, $h'(n)$ merupakan biaya aktual minimum dari simpul n menuju tujuan. Jaminan ini diperlakukan agar nantinya algoritma A* dapat dijamin menghasilkan solusi yang dinilai optimal.

Sebelum menganalisis masing-masing heuristik, perlu dipahami terlebih dahulu model biaya dari puzzle ini. Jika melihat implementasi fungsi Slide kami, nanti saat *user* bergerak ke suatu arah, *user* akan bergerak secara terus melewati beberapa *tile* hingga nanti berhenti di depan tembok. Biaya gerakan yang diambil akan diambil dari akumulasi biaya menginjak ke sebuah *tile* yang ingin dilewati. Hal tersebut membuat biaya yang dibayar tetaplah proporsional dengan jumlah *tile* yang dilintasi.

3.1.1 Analisis Admissibility H1 (Manhattan Distance)

Biaya untuk menghitung heursitik H1 dapat melihat formula di bawah ini,

$$h_1(n) = |row_n - row_{goal}| + |col_n - col_{goal}|$$

Karena arah gerak pada *puzzle* bersifat 4 arah saja (atas, bawah, kiri, kanan), untuk berpindah dari suatu posisi ke posisi tujuan, *user* harus melewati seminimalnya Δrow *tile* secara vertikal & Δcol secara horizontal. Kedua komponen tersebut tidak dapat dipersingkat lagi, alias ***bare minimum***. Hal tersebut membuat pada jalur mana pun dari suatu posisi ke tujuan, jumlah *tile* yang dilintasi paling sedikitnya sebesar jarak Manhattan.

Mekanisme pergerakan dari *game* ini membuat heuristik menjadi *overestimate*. Hal ini dikarenakan biaya akan dihitung per *tile* yang akan dilewatinya. Jika nanti *user* terpaksa melewati *tile* tambahan karena satu dan lain hal, biaya aktual akan menjadi lebih besar dari *cost* Manhattan. Selain itu, H_1 juga mengabaikan kewajiban mengunjungi *checkpoint* secara berurutan yang seharusnya akan membuat yang harus dilewati bukanlah yang optimal. Maka dari itu,

$h_1(n) \leq h'(n)$*; untuk semua simpul n, sehingga H_1 bersifat* ***admissible****.*

[Bagian ini diisi dengan argumen formal atau informal mengapa H1 admissible: h(n) tidak pernah melebihi cost aktual minimum, dengan memperhatikan mekanisme sliding yang melompati banyak tile sekaligus dalam satu gerakan]

3.1.2 Analisis Admissibility H2 (Euclidean Distance)

Biaya untuk menghitung heursitik H2 dapat melihat formula di bawah ini,

$$h_2(n) = \lfloor \sqrt{(row_n - row_{goal})^2 + (col_n - col_{goal})^2} \rfloor$$

Kalau kita melihat sifat dasar dari geometri yang menjelaskan jarak Euclidean antara dua titik akan lebih kecil atau sama dengan jarak Manhattan antara dua titik yang sama, dengan sama jika dalam satu garis lurus di sesuai arah yang ada (atas, bawah, kiri, kanan) dan lebih kecil jika arahnya tidak sesuai dengan 4 itu.

Dengan adanya *floor* untuk perhitungan H_2 membuatnya semakin mengurangi nilai estimasinya (meskipun hanya < 0.5), sehingga semakin memperkuat sifat *underestimate* yang dihasilkannya. Karena H_1 sudah kami argumen bahwasannya itu *admissible*, dan karena $h_2(n) \leq h_1(n)$, maka akan berbentuk seperti

$h_2(n) \leq h_1(n) \leq h'(n)$*; untuk semua simpul n, sehingga H_2 bersifat* ***admissible*** *juga.*

[Bagian ini diisi dengan argumen admissibility H2: karena Euclidean <= Manhattan dan Manhattan sudah admissible, maka Euclidean juga admissible]

3.1.3 Analisis Admissibility H3 (Chebyshev Distance)

Biaya untuk menghitung heursitik H3 dapat melihat formula di bawah ini,

$$h_3(n) = MAX(|row_n - row_{goal}|, |col_n - col_{goal}|)$$

Dalam fungsi MAX tersendiri, hasil dari fungsi tersebut akan lebih kecil dibanding perjumlahan di antara kedua posisi.

$$MAX(a,\ b) \leq a + b$$

Ingat kalau nilai a & b yang digunakan dalam contoh diatas sebenarnya hanyalah komponen dalam perhitungan Manhattan Distance yang seperti di formula di atasnya lagi. Mengingat H_1 telah terbukti *admissible*, maka seharusnya

$h_3(n) \leq h_1(n) \leq h'(n)$*; untuk semua simpul n, sehingga* H_3 *bersifat* ***admissible*** *juga.*

[Bagian ini diisi dengan argumen admissibility H3: karena Chebyshev <= Manhattan (max(|dr|,|dc|) <= |dr|+|dc|) dan Manhattan sudah admissible, maka Chebyshev juga admissible]

3.2 Perbandingan Teoritis Algoritma

Simpelnya, ketiga algoritma pencarian hanya memiliki perbedaan dari fungsi untuk perhitungan prioritas eksplorasinya nanti,

UCS: $f(n) = g(n)$ // Biaya aktual simpul awal ke simpul n. $GBFS$: $f(n) = h(n)$ // Biaya estimasi heuristik A *: $f(n) = g(n) + h(n)$ // Gabungan antara kedua			
	UCS	**GBFS**	**A***
Time Complexity	$O(b^{(C^*/\varepsilon)})$	$O(b^m)$	$O(b^d)$
Space Complexity	$O(b^{(C^*/\varepsilon)})$	$O(b^m)$	$O(b^d)$

1. UCS

   UCS atau *Uniform Cost Search* akan melakukan eksplorasi *node* berdasarkan biaya jalur terkecil dari *starting node* saja. Algoritma ini akan menjamin *completeness* selama biaya tiap langkahnya bernilai positif ($\varepsilon > 0$), dan menjamin *optimality* karena simpul tujuan baru diekspansi ketika memiliki biaya *g(n)* terkecil.

   Namun, UCS kurang optimal karena tidak memiliki eksplorasi yang terarah. Dia akan mengeksplorasi setiap simpul dengan biaya yang lebih kecil saja. Bisa saja dia mengeksplorasi *node* lain padahal *node* tujuan juga ada hanya satu langkah lagi, namun gagal dideteksi karena hanya melihat *cost* untuk melompat.

2. GBFS

   GBFS atau *Greedy Best-First Search* akan melakukan eksplorasi *node* berdasarkan nilai *heuristik* terkecil yang dimiliki oleh calon *node*. Strateg ini akan menghasilkan pencarian yang cepat dalam kasus-kasus tertentu karena selalu mengarah ke titik tujuan. Namun, hal tersebut tidak menjamin *completeness* maupun *optimality*. GBFS sebenarnya sering menghasilkan jalur yang tidak optimal jika heuristiknya itu *fooling* (tidak menempel misalnya tapi deket banget).

3. A*

   A* search merupakan campuran dari UCS & GBFS yang menghitung biaya lompat dan nilai heuristik yang dimiliki *node* tersebut. Selama heuristik tersebut *admissible*, A* menjamin *completeness* dan *optimality*. Keunggulan dari

algoritma ini ada di efisiensi dari eksplorasi dengan memanfaatkan nilai heuristik untuk memiliki arah ke *node* tujuan.

[Bagian ini diisi dengan tabel atau uraian perbandingan UCS, GBFS, dan A mencakup: definisi f(n), completeness, optimality, time complexity, dan space complexity masing-masing algoritma]*

3.3 Analisis Kompleksitas pada Ice Sliding Puzzle

Terdapat tiga faktor utama sebarnya yang akan mempengaruhi kompleksitas dari pencariannya, yaitu

1. Ruang *Board*

   State pada puzzle ini didefinisikan oleh `State{Row, Col, CheckpointIdx}`. Dengan *board* yang berukuran $N \times M$ ditambah dengan K *checkpoint* yang harus dilewati, total *space* akan sebanyak $|S| = N \times M \times (K + 1)$. Bagian $K + 1$ muncul karena `CheckpointIdx` yang mendefinikasikan seberapa banyaka *checkpoint* yang harus dilewati. Karena di papan sebenarnya ada tembok di dalamnya, sebenarnya formula tersebut terlalu besar dengan jumlah nyatanya.

2. Mekanisme Pergerakan

   Dengan adanya mekanisme yang akan selalu bergerak hingga mengenai tembok atau tidak valid, terdapat sebuah dampak terhadap kompleksitas pencarian ini. **Pertama**, *branching factor* tidak sepenuhnya akan sama dengan arah yang dapat dipilih oleh *user* yang dimana nilainya 4. Hal tersebut dikarenakan adanya *lava*, *checkpoint* yang seharusnya belum boleh dilewati, dan lain hal. **Kedua**, *depth* solusi umumnya akan lebih kecil karena sebuah gerakan akan melewati banyak *tile* secara serentak. Alhasil, *depth* dari pohon pencarian akan menjadi lebih dangkal. **Ketiga**, variasi biaya membuat perhitungan UCS & A* lebih disarankan dibandingkan *rely* dengan heuristik sepenuhnya karena dapat saja lebih merugikan karena hal tersebut.

3. Aturan ke Checkpoint

   Checkpoint harus dilewati sesuai dengan urutan dari *list of checkpoints*. Alhasil, ruang pencaraian akan lebih dipangkas karena hal tersebut dikarenakan akan lebih banyak gerakan yang tidak *valid* akibatnya. Namun, sebenarnya ini membuat *depth* dari pohon *search* jadi lebih dalam dikarenakan harus mengeksplorasi *new path* yang dapat memenyesuaikan aturan tersebut. Selain itu juga, heuristik ini menjadi krisis jika terlalu memiliki *checkpoint* yang belum dilewati. Mengapa? Karena heuristik ini hanya memikirkan jarak menuju satu *tile tujuan* tanpa ada *forward thinking* terhadap *node-node* lainnya.

[Bagian ini diisi dengan diskusi tentang bagaimana karakteristik Ice Sliding Puzzle seperti mekanisme sliding, ukuran papan yang variatif, dan constraint angka berurutan mempengaruhi kompleksitas pencarian masing-masing algoritma dalam praktik]

Bab IV — Implementasi Program

4.1 Bahasa Pemrograman dan Struktur Program

Kami menggunakan bahasa pemrograman Go (Golang) untuk pengimplementasiannya. Pemilihan bahasa Go didasari dengan efisiensi dari bahasanya. Go tersendiri merupakan bahasa yang efisien dikarenakan bahasa yang dikompilasi sehingga menghasilkan *binary* yang cepat dan efisien. Algoritma *searching* yang diimplementasi untuk UCS, GBFS, dan A* melibatkan operasi *priority queue* yang relatif berat yang dimana juga hal tersebut Go mampu menanganinya dengan baik. Selain itu, Go juga menyeiakan paket container/heap yang langsung dapat digunakan untuk nantinya mengimplementasikan *priority queue*.

```
src/
├── main.go
├── gui.go
└── puzzle/
    ├── board.go
    ├── state.go
    ├── heuristic.go
    └── solver.go
```

- src/main.go

  File ini menjadi titik masuk ari program. Saat nanti *run*, program akan memeriksa argumen baris perintah. Jika nantinya ditemukan *flag* –gui, keberjalanan *run* akan dilakukan melalui GUI (`runGUI()`), seangkan dalam *default* program akan berjalan di CLI (`runCLI()`). *File* ini akan mengurus *interaction* dengan *user* via *terminal* or GUI. Fiturnya akan berupa *read input file*, memilih algoritma & heuritik, penampilan dari solusi (step-by-step), dan ada *replay system*.

- src/gui.go

  File ini hanya untuk pengimplementasian dari *UI* dengan menggunakan *library* Fyne. GUI akan dibagi menjadi *control panel* yang ada di kiri untuk pemilihan *file*, algoritma, heuristik, dan *run program*, serta papan permainannya yang menjadi tempat hasil *render* dari *board*-nya. GUI juga dilengkapi dengan fitur *auto-play*.

- src/puzzle/board.go

  File ini mendefinisikan struktur *Board* yang akan menyimpan seluruh informasi *board* nya dengan ukuran $N \times M$. Informasi yang disimpan berupa jenis *tile*, *cost* untuk gerak, posisi awal, posisi tujuan, dan daftar *checkpoint*.

  Terdapat beberapa fungsi pendukung, yaitu ParseBoard untuk membaca *file text* dari *input*, memvalidasikannya, dan mengembalikan objek *Board* yang berisikan informasi dari *input*. Selain itu, juga terdapat fungsi IsGoal untuk mengecek kondisi tujuan. Dan ada fungsi *print* untuk menghasilkan teks papan pada bagian tertentu.

- src/puzzle/state.go

  File ini mendifinisikan *State* yang akan merpresentasikan keadaan *user* yang berisikan posisi baris & kolom, serta indeks *checkpoint* yang sudah mereka

kunjungi. Terdapat *function* utama di *file ini*, yaitu Slide. *Function* tersebut dirancang untuk mensimulasikan pergerakan *user* di atas es ke sebuah arah gerak (atas, bawah, kiri, kanan). *User* akan terus bergerak, karena licin, ke arah yang dipilih hingga menabrak tembok, keluar dari *board*, atau mati karena kena *lava*. Fungsi sini menangani pengumpulan *checkpoint* juga secara berurutan dan mengembalikan dalam bentuk *state* baru, total biaya untuk gerak, dan validitas dari pergerakan (*dead or alive*)

- src/puzzle/heuristic.go

  File ini akan menyiapkan tiga fungsi heuristik yang akan digunakan oleh algoritma A* & GBFS.

  - **H1: Manhattan Distance**, yaitu menghitung selisih mutlak dari baris & kolom. $|\Delta Row| + |\Delta Col|$.
  - **H2: Euclidean Distance**, yaitu jarak lurus yang dibulatkan ke bawah menjadi *integer*. $\sqrt{\Delta Row^2 + \Delta Col^2}$
  - **H3: Chebyshev Distance**, yaitu nilai maksimum di antara selisih mutlak baris & kolom (mirip ReLU). $MAX(|\Delta Row|, |\Delta Col|)$

- src/puzzle/solver.go

  File ini menjadi inti dari program yang mengimplementasikan ketiga algoritma *search*. Ketiganya akan dibangun dalam sebuah fungsi *solve* yang akan menerima *input board* dan fungsi prioritas. Fungsi ini menggunakan *priority queue* berbasis *min-heap* untuk mengeksplorasi *state* secara berurutan sesuai dengan prioritasnya, dengan mekanisme *visited list*.

  - **SolveUCS**, menetapkan prioritas sama dengan biaya aktual, jadi akan *explore* jalur dengan biaya terendah dulu.
  - **SolveGBFS**, menetapkan prioritas sama dengan heuristik, jadi akan *explore* sesuai perkiraan jarak ke tujuan.
  - **SolveAStar**, menetapkan prioritas sebagai biaya aktual + heuristik, untuk memberikan solusi yang optimal (if heuristik bersifat *admissible*)

[Bagian ini diisi dengan bahasa pemrograman Go yang dipilih, alasan pemilihan, dan penjelasan struktur folder: src/puzzle/ berisi board.go, state.go, heuristic.go, solver.go; src/ berisi main.go dan gui.go]

4.2 Representasi State

State dalam implementasi kami akan direpresentasikan oleh struktur *State* sebagai berikut.

```go
type State struct {
    Row, Col int
    CheckpointIdx int
}
```

Struktur tersebut akan menyimpan tiga informasi dengan tujuan mendeskripsikan kondisi permainan pada suatu waktu.

- **Row & Col** untuk menyimpan posisi ari *user* pada *board game* saja yang akan dinyatakan dalam indeks baris dan kolom. Dua nilai tersebut kami nilai cukup untuk menyimpan karena *board* berbentuk 2D.

- **CheckpointIDX** itu untuk menyimpan pengumpulan *checkpoint* yang telah diselesaikan. Nilainya akan berupa indeks *checkpoint* berikutnya yang harus dikunjungi. Nanti kalau jumlah *chekpoint* yang ingin disamper dan sudah disamper sama, berarti boleh berhenti.

[Bagian ini diisi dengan penjelasan struct State yang menyimpan Row, Col, dan CheckpointIdx sebagai representasi posisi pin dan progress checkpoint yang sudah dilalui]

4.3 Mekanisme Sliding

Mekanisme pergerakan diimplementasi dalam fungsi Slide. Fungsi ini simpelnya akan menerima *board*, *state*, dan *direction*, lalu bakal menggerakan pin secara lengkap hingga pin berhenti atau sampai gerakan jadi tidak valid.

Secara rinci, nantinya pin akan bergerak *tile* demi *tile* ke arah yang dipilih secara berulang dalam sebuah loop (karena kan licin). Di tiap iterasinya, program akan menghitung koordinat *tile* berikutnya dengan menambahkan sesuai arah yang dipilih, lalu akan validasi kondisi *tile* tersebut sebelum menentukan pin dapat melangkah atau tidak.

Pin akan berhenti bergerak ketika *tile* berikutnya adalah tembok. Pin akan tetap berhenti di posisi terakhir yang valid sebelum di dalam tembok. Selain itu, pergerakan akan dapat dianggap tidak valid karena beberapa *cases*, yaitu

1. Keluar *board* kalau nanti koordinat *tile* berikutnya di luar dimensi dari *board*nya tersendur sebelum dia menemukan tembok untuk berhenti.
2. Melewati *tile* lava yang nanti saat menginjakny, *pin* akan “mati” karena telah melewatinya.
3. Tidak bergerak sama sekali, kalau dia memilih untuk bergerak ke sebuah arah namun ternyata tidak bergerak sama sekali.

Saat *pin* melewati *tile* angka, program akan memeriksa apakah angka tersebut sesuai dengan *checkpoint* yang sedang ditunggu. Kalau nanti cocok, checkpointIdx bakal dinaikin satu karena sudah berhasil melewati sebuah *checkpoint*. Kalau lebih besar, *pin* tidak jadi bergerak karena tidak boleh melewatinya dahulu. Kalau lebih kecil, *pin* boleh bergerak karena sudah melewatinya.

Setiap *tile* yang diinjak selama satu gerakan akan menaikan *cost* sesuai nilai yang ada di *tile* tersebut. Biaya tersebut bakal diakumulasi dan dikembalikan ke SlideResult (hasil return function) untuk nanti dipakai menghitung GCost.

Di akhir, kalau gerakannya valid, *function* akan mengembalikan SlideResult yang berisikan *state* baru, total biaya, dan *flag* yang menyatakan gerakan *valid*.

[Bagian ini diisi dengan penjelasan implementasi fungsi Slide di state.go: pin bergerak tile demi tile ke arah yang dipilih, berhenti saat menabrak TileWall atau batas papan, game over jika keluar papan tanpa rintangan atau melewati TileLava, serta penanganan tile angka checkpoint]

4.4 Perhitungan Cost

Biaya sebuah gerakan sebenarnya dihitung dalam fungsi Slide dengan akumulasi dari nilai dari *tile* yang mereka lewati saat melakukan pergerakan. Algoritma cukup simpel dengan terus menambahkan biaya untuk melewati/menginjak *tile* tersebut dengan *tile-tile* selanjutnya yang akan dilewati, termasuk juga *tile* terakhir.

[Bagian ini diisi dengan penjelasan bagaimana cost gerakan dihitung di Slide: menjumlahkan board.Costs[curRow][curCol] setiap tile yang dilewati selama sliding, tidak termasuk posisi awal, tetapi termasuk posisi berhenti]

4.5 Format Input

File dari *input* akan berbentuk *text* (.txt) yang dibaca oleh fungsi ParseBoard. File tersebut terdiri atas 3 bagian yang dibaca secara urutan.

a. Baris Pertama, yaitu dua bilangan yang dipisahkan spasi yang menyatakan ukuran dari *board* (N: Baris(Row) & M: Column(Col))
b. Baris $N \times M$ berikutnya akan meminta konten dari *board* yang ingin dijadikan untuk permainan. Rincian detail dari *board* dapat melihat dari tabel di bawah.

Karakter	Konstanta	Makna
*	TilePath	Jalur yang dapat dilewati
X	TileWall	Tembok
L	TileLava	Lava (berbahaya)
Z	TileStart	Posisi awal dari *user*
O	TileGoal	Posisi tujuan
0—9		*Checkpoint* yang harus dikunjungi oleh *user* secara berurut (0, 1, 2, …, 9)

c. Baris $N \times M$ berikutnya lagi akan meminta detail dari biaya tiap *tile* untuk nanti diakumulasikan saat melakukan pergerakan.

[Bagian ini diisi dengan penjelasan format file .txt yang dibaca oleh ParseBoard di board.go: baris pertama N M, N baris karakter papan, N baris cost integer; serta validasi yang dilakukan seperti jumlah checkpoint, posisi start/goal, dan ukuran papan]

4.6 Format Output

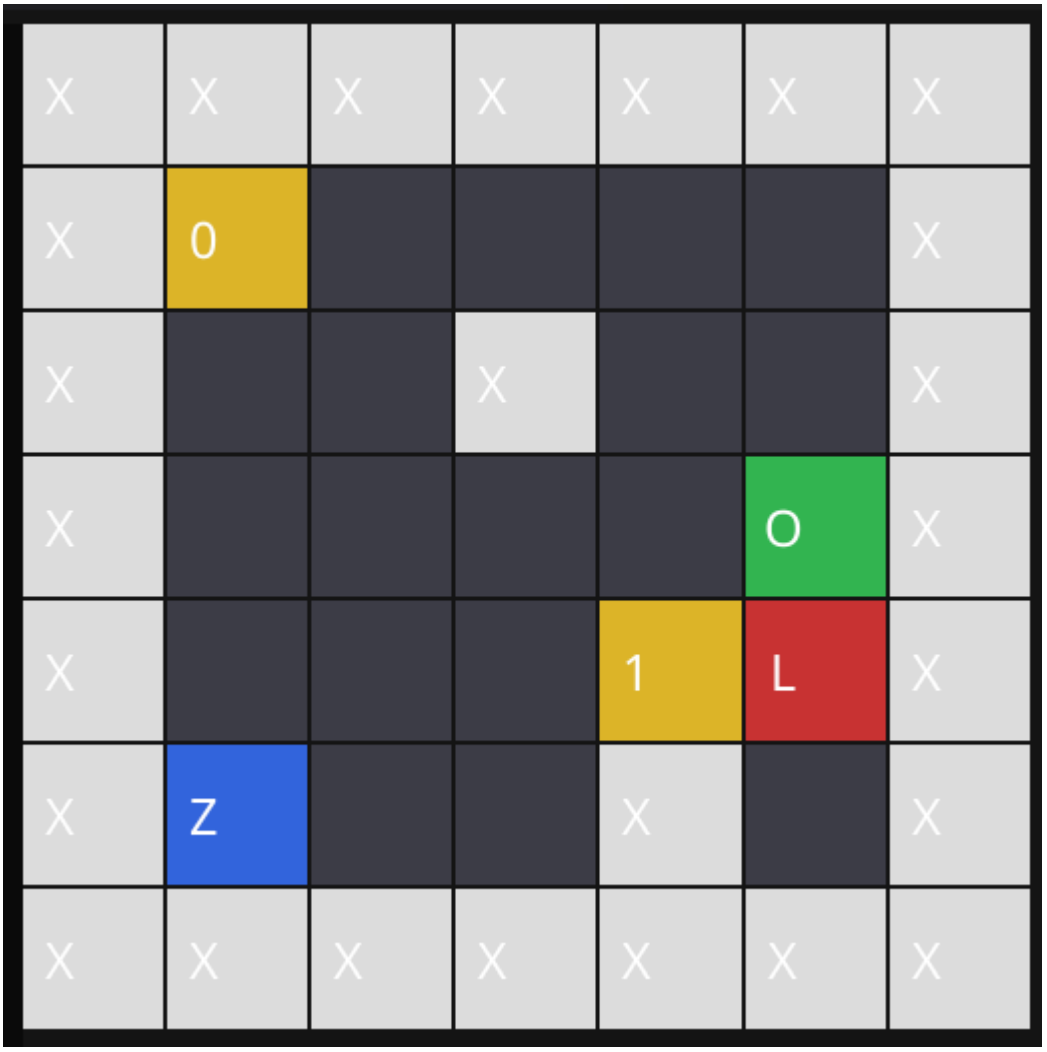
Program akan menghasilkan *output* dalam dua bentuk, yaitu tampilan CLI dan GUI yang disesuaikan dengan keinginan dari *user*. Dalam bentuk CLI, *output* simpelnya akan memberikan sebuah ringkasan dari *solusi* yang akan memaparkan solusi dan biaya. Selain itu, nanti langkah yang diambil akan divisualisasikan dengan memberikan bentuk *board* di awal mulai permainan, step yang diambil, hingga nanti berakhir. Diakhir, juga akan ada bagian statistik dari iterasi yang menampilkan waktu dan banyak iterasinya tersendiri. Selain itu juga, ada fitur *replay* yang kami implementasi yang dapat melihat tiap langkah yang pernah diambil.

[Bagian ini diisi dengan penjelasan format output program: urutan gerakan solusi, total cost, visualisasi papan step-by-step via PrintWithActor, jumlah iterasi, waktu eksekusi dalam ms, fitur playback CLI dengan arrow key, dan opsi simpan solusi ke .txt]

Bab V — Hasil Percobaan

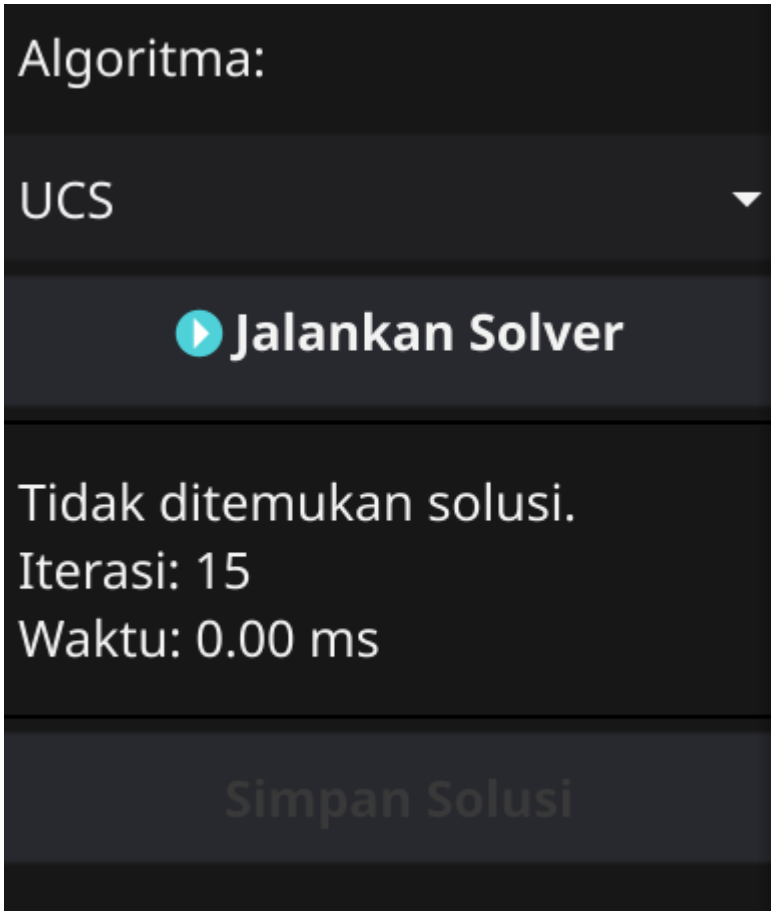
5.1 Contoh Test Case 1

Input



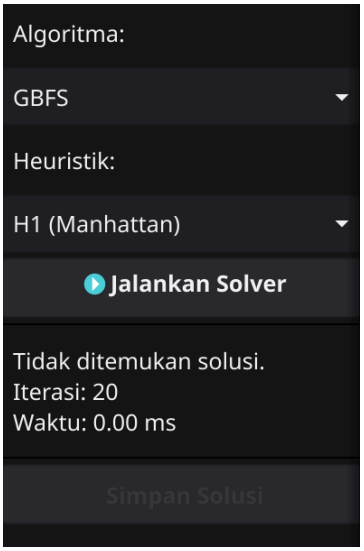
Gambar 5.1.1 input test case 1

Output — UCS

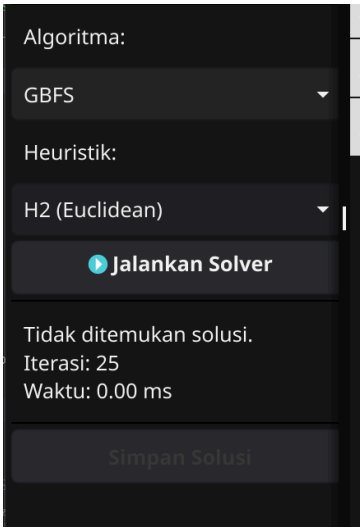


Gambar 5..1.2 Output UCS test case 1

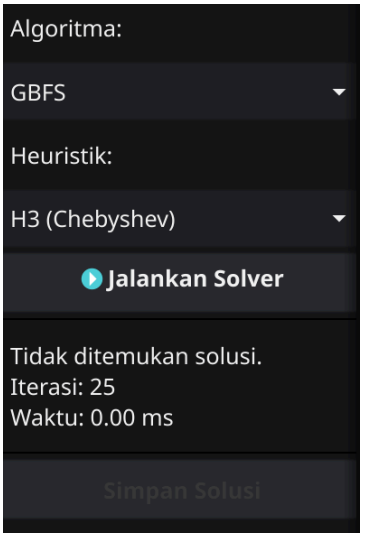
Output — GBFS



Gambar 5.1.3 Output GBFS H1(test case 1)

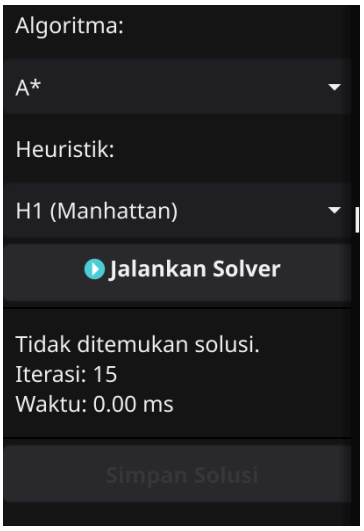


Gambar 5.1.4 Output GBFS H2(test case 1)

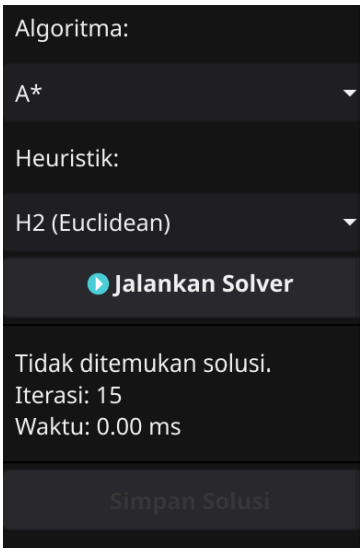


Gambar 5.1.5 Output GBFS H3(test case 1)

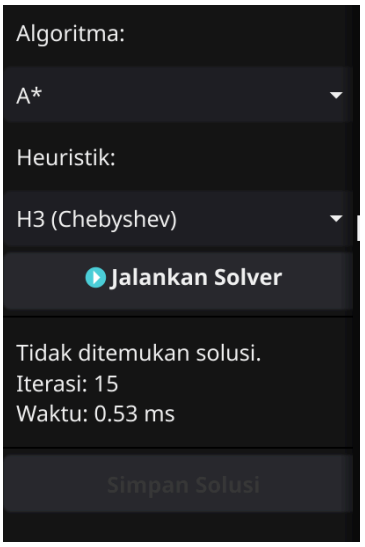
Output — A*



Gambar 5.1.6 Output A* H1(test case 1)



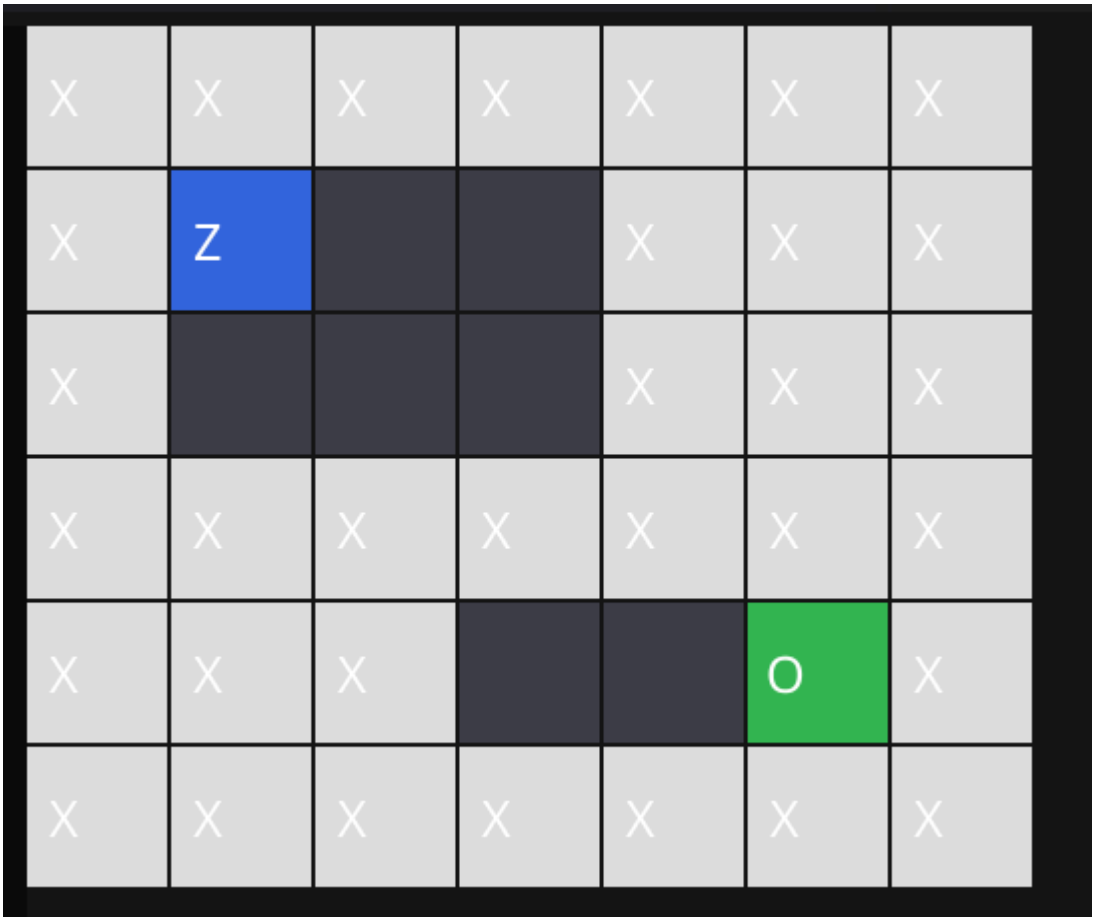
Gambar 5.1.7 Output A* H1(test case 1)



Gambar 5.1.8 Output A* H1(test case 1)

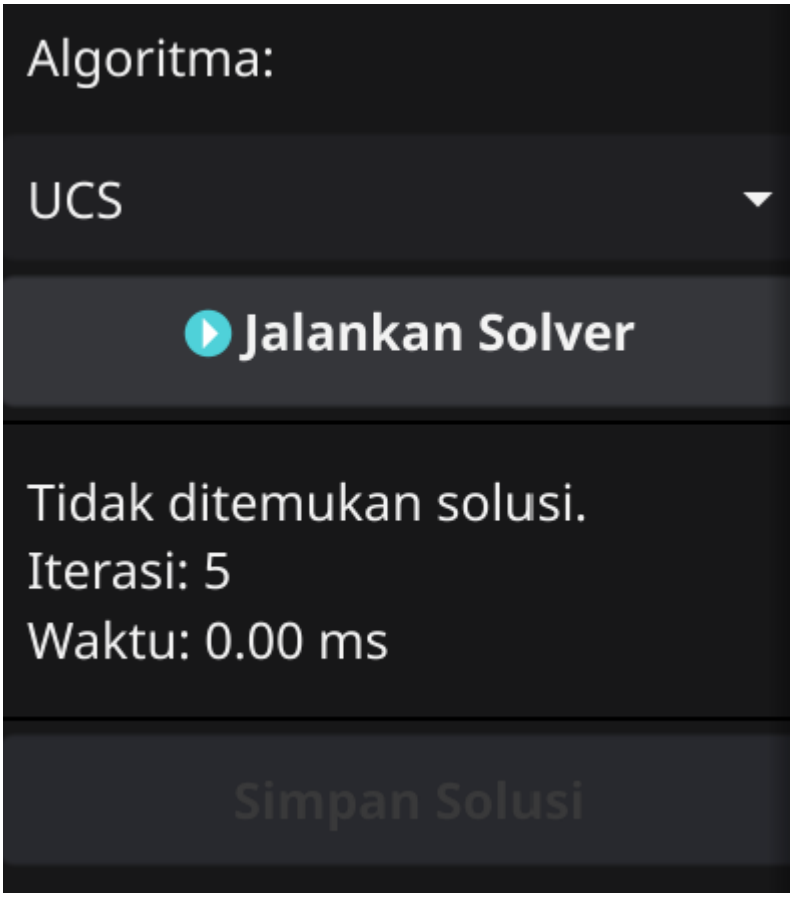
5.2 Contoh Test Case 2

Input



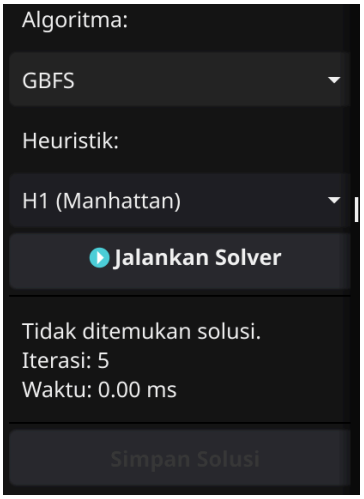
Gambar 5.2.1 input test case 2

Output — UCS



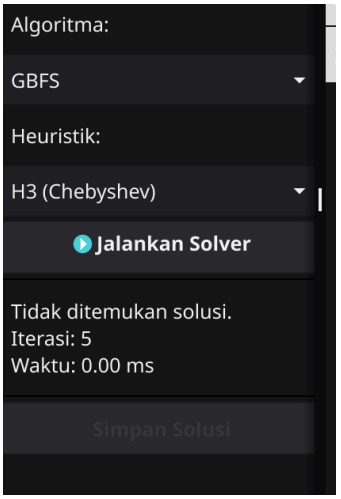
Gambar 5.2.2 Output UCS test case 2

Output — GBFS



Gambar 5.2.3 Output GBFS H1(test case 2)

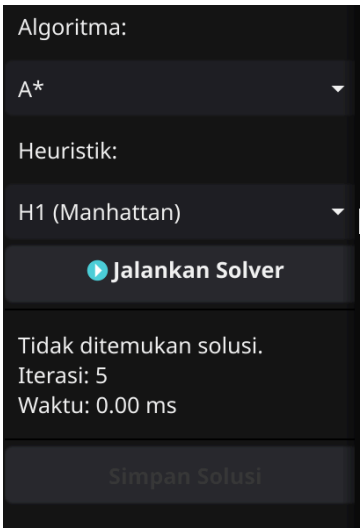
Gambar 5.2.4 Output GBFS H2(test case 2)



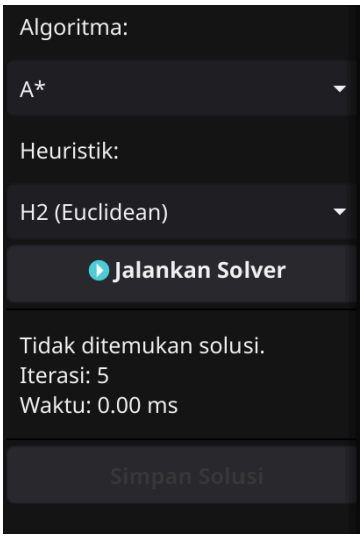
Gambar 5.2.5 Output GBFS H3(test case 2)

[Bagian ini diisi dengan screenshot output GBFS untuk test case 2: solusi gerakan, cost, jumlah iterasi, dan waktu eksekusi]

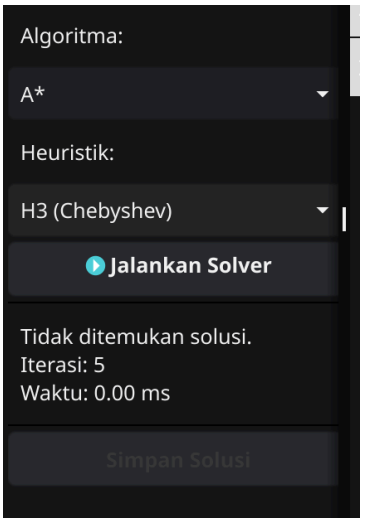
Output — A*



Gambar 5.2.6 Output A* H1(test case 2)



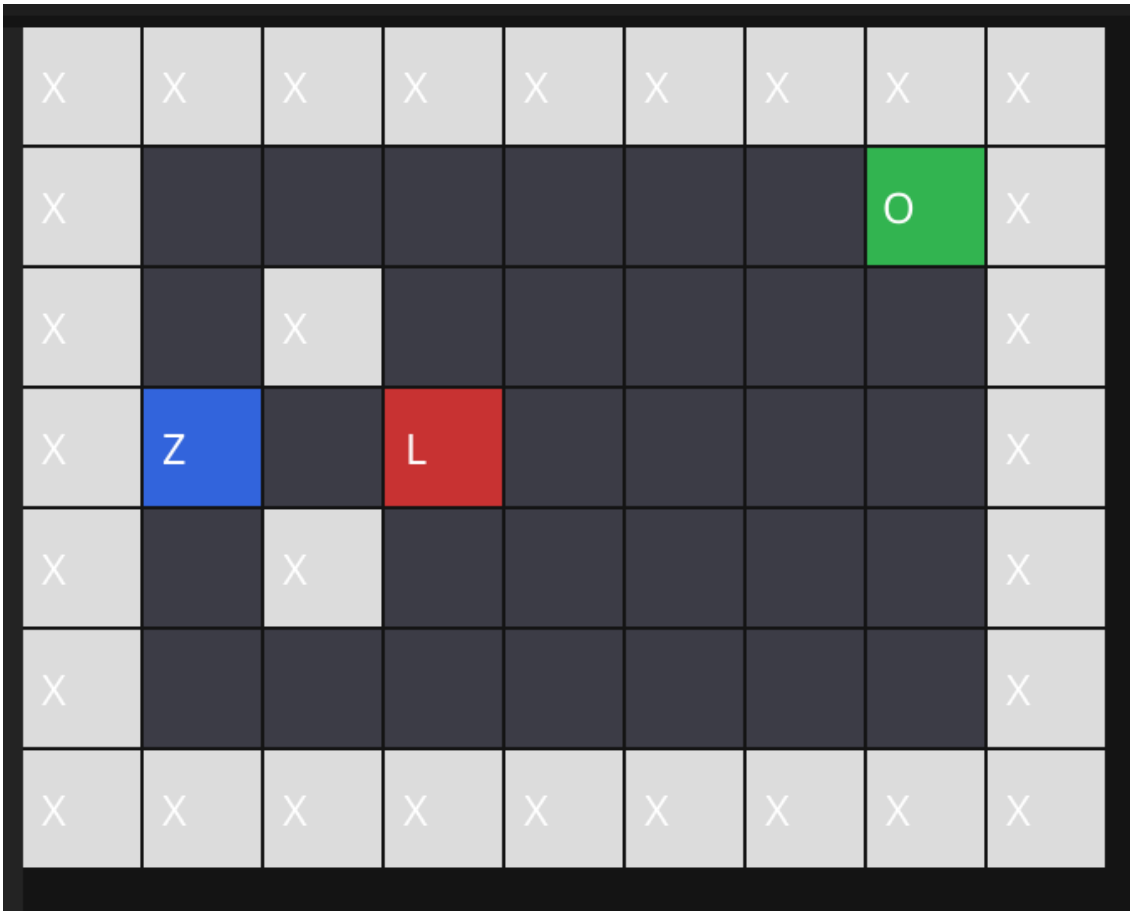
Gambar 5.2.7 Output A* H2(test case 2)



Gambar 5.2.8 Output A* H3(test case 2)

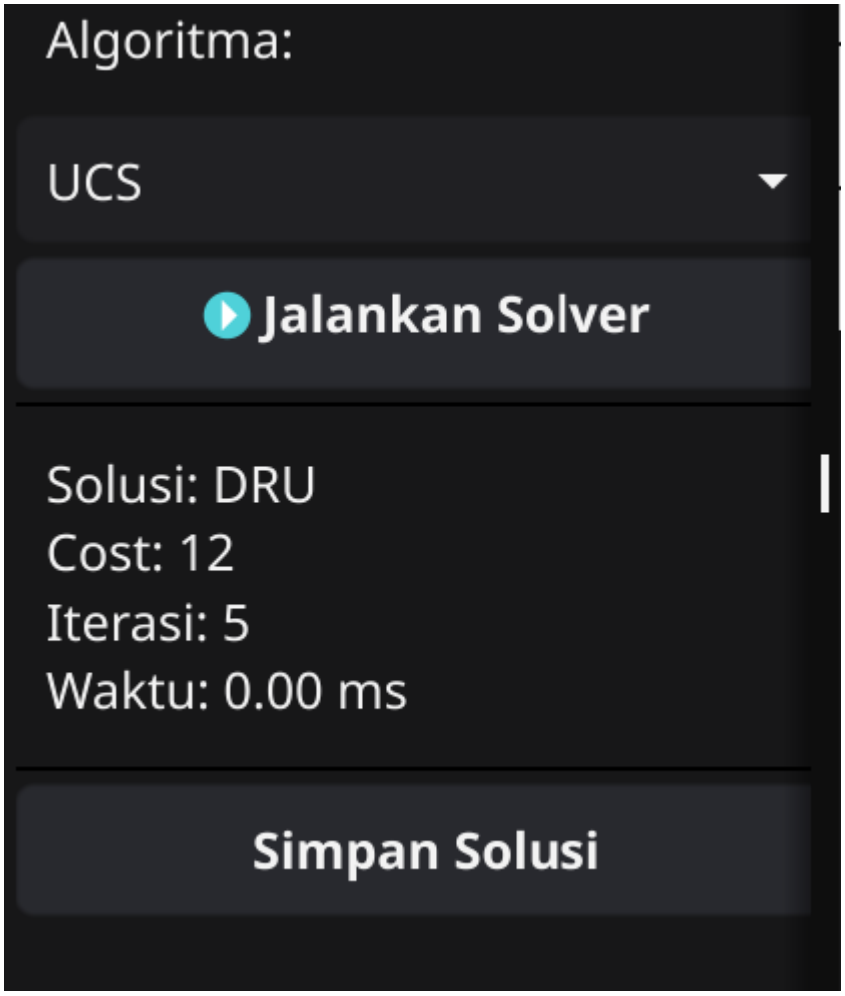
5.3 Contoh Test Case 3

Input



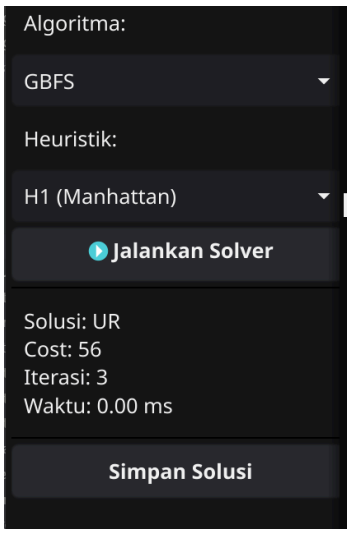
Gambar 5.3.1 input test case 3

Output — UCS

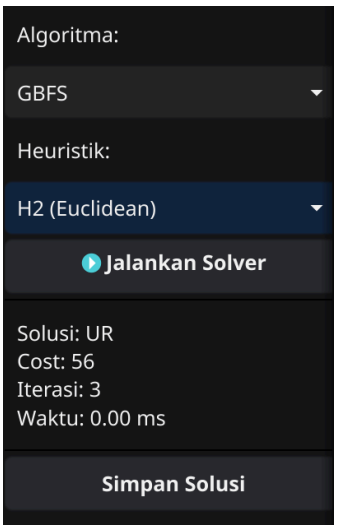


Gambar 5.3.2 Output UCS test case 3

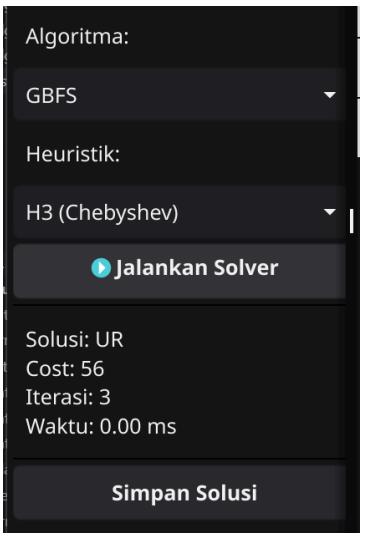
Output — GBFS



Gambar 5.3.3 Output GBFS H1(test case 3)

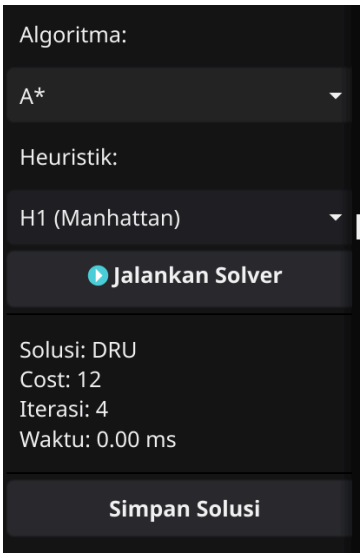


Gambar 5.3.4 Output GBFS H2(test case 3)

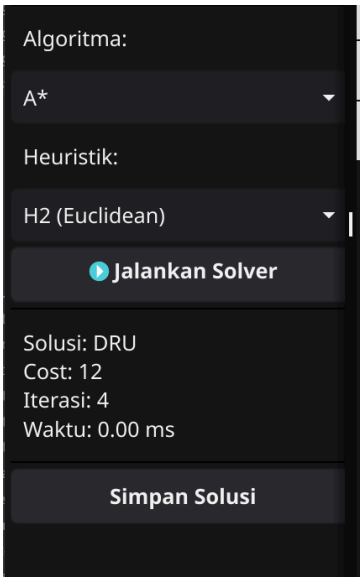


Gambar 5.3.5 Output GBFS H3(test case 3)

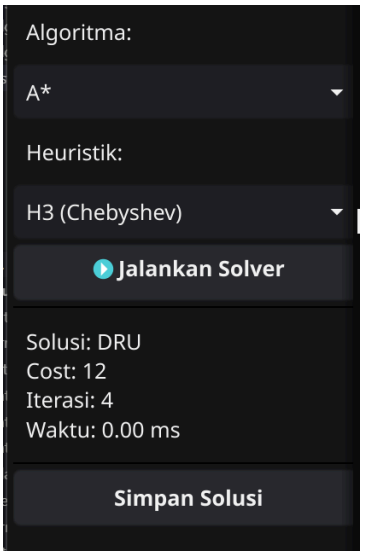
Output — A*



Gambar 5.3.6 Output A* H1(test case 3)



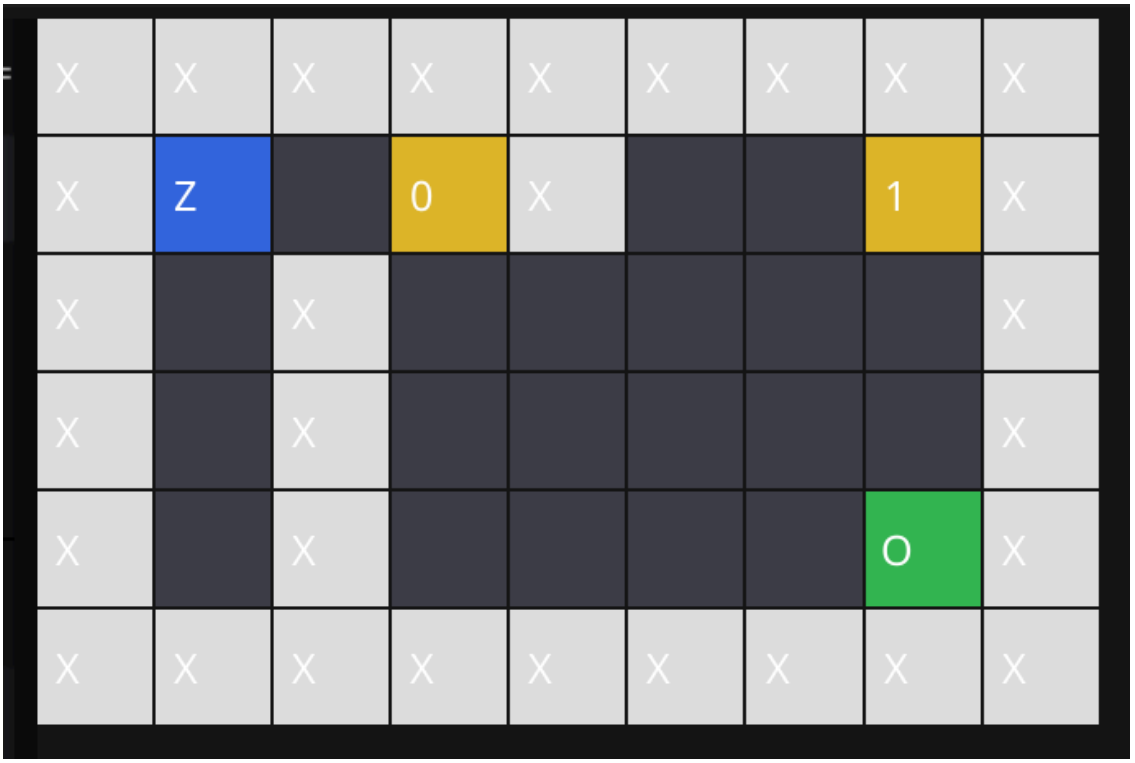
Gambar 5.3.7 Output A* H2(test case 3)



Gambar 5.3.8 Output A* H3(test case 3)

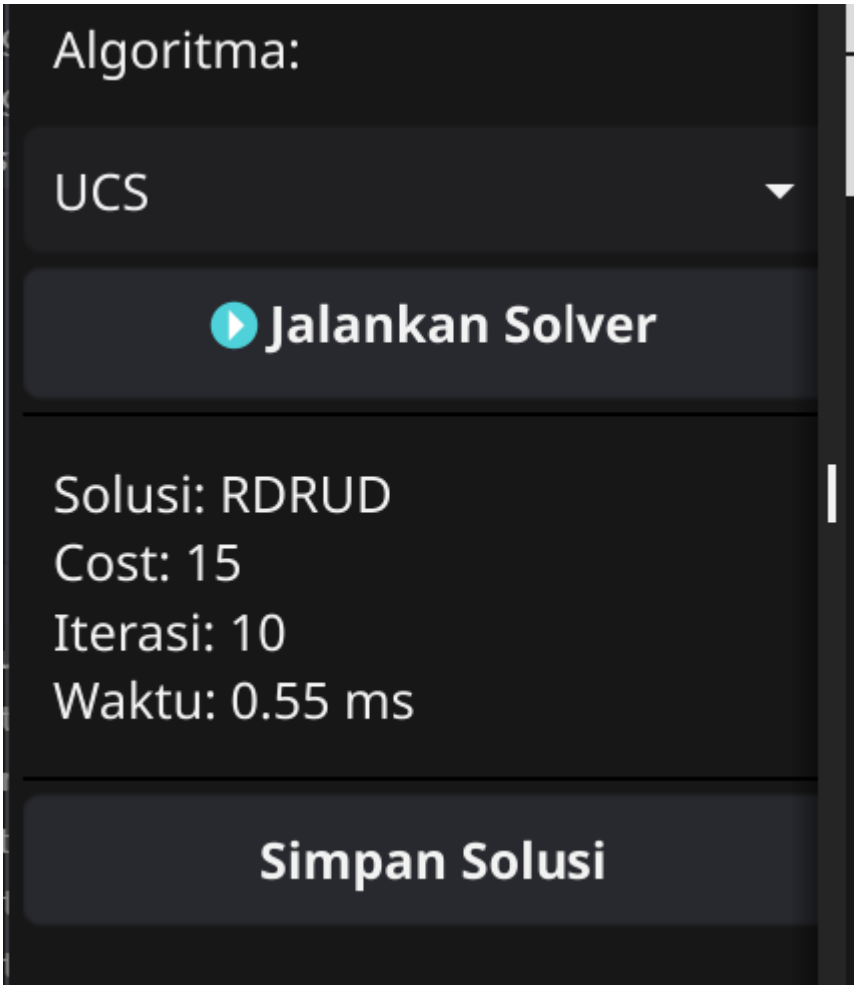
5.4 Contoh Test Case 4

Input



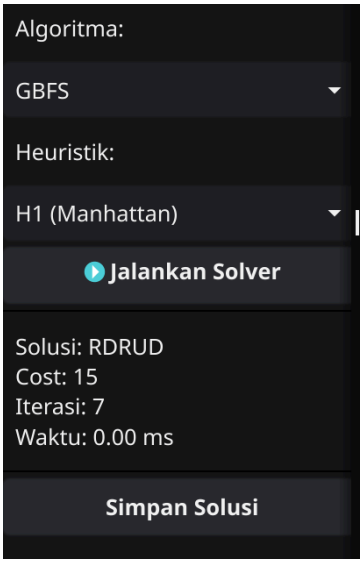
Gambar 5.4.1 input test case 4

Output — UCS



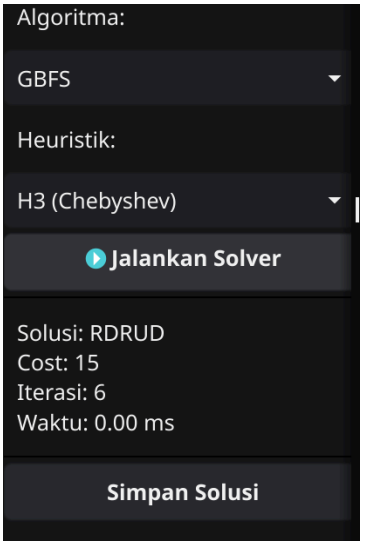
Gambar 5.4.2 Output UCS test case 4

Output — GBFS



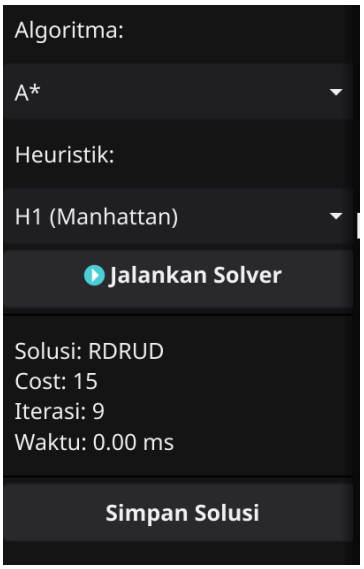
Gambar 5.4.3 Output GBFS H1(test case 4)

Gambar 5.4.4 Output GBFS H2(test case 4)



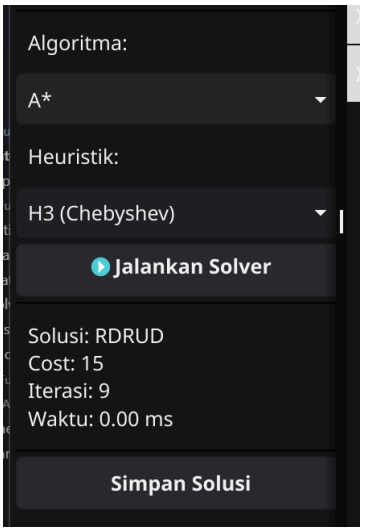
Gambar 5.4.5 Output GBFS H3(test case 4)

Output — A*



Gambar 5.4.6 Output A* H1(test case 4)

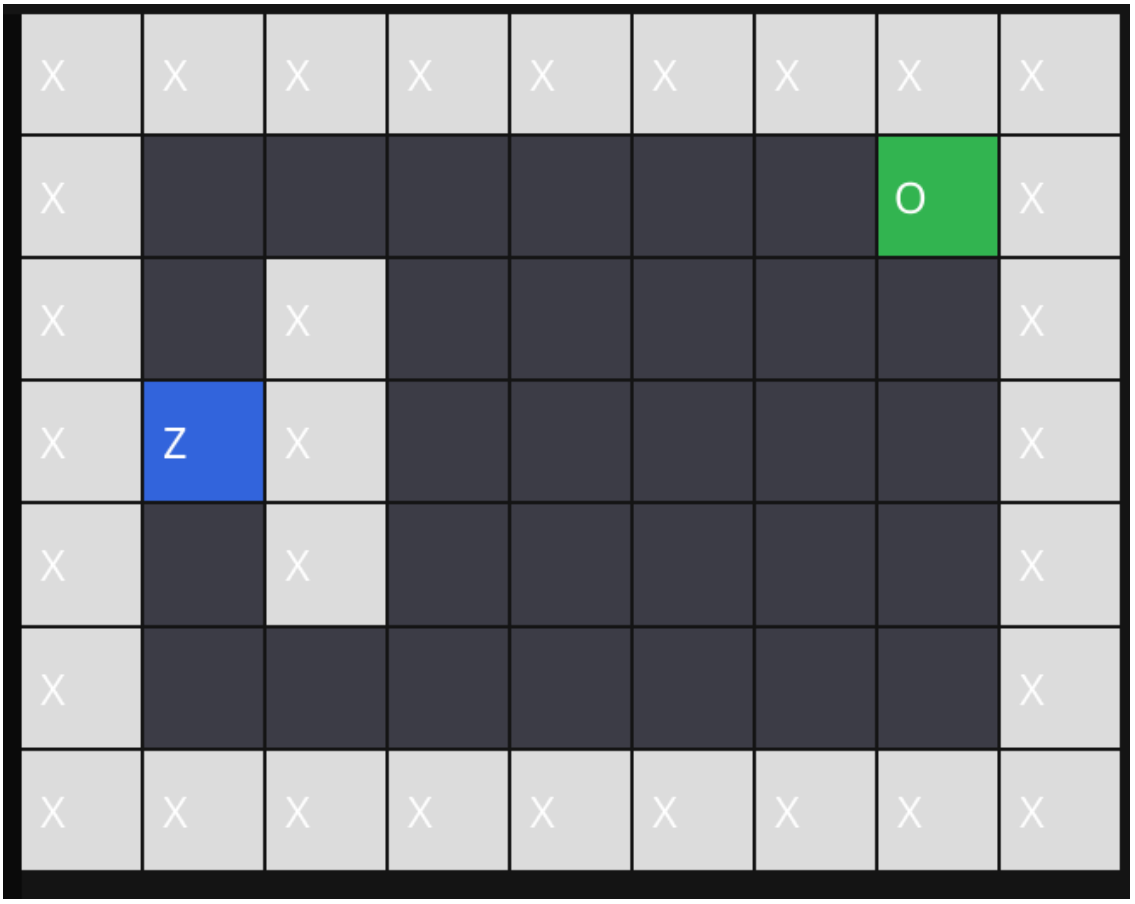
Gambar 5.4.7 Output A* H2(test case 4)



Gambar 5.4.8 Output A* H3(test case 4)

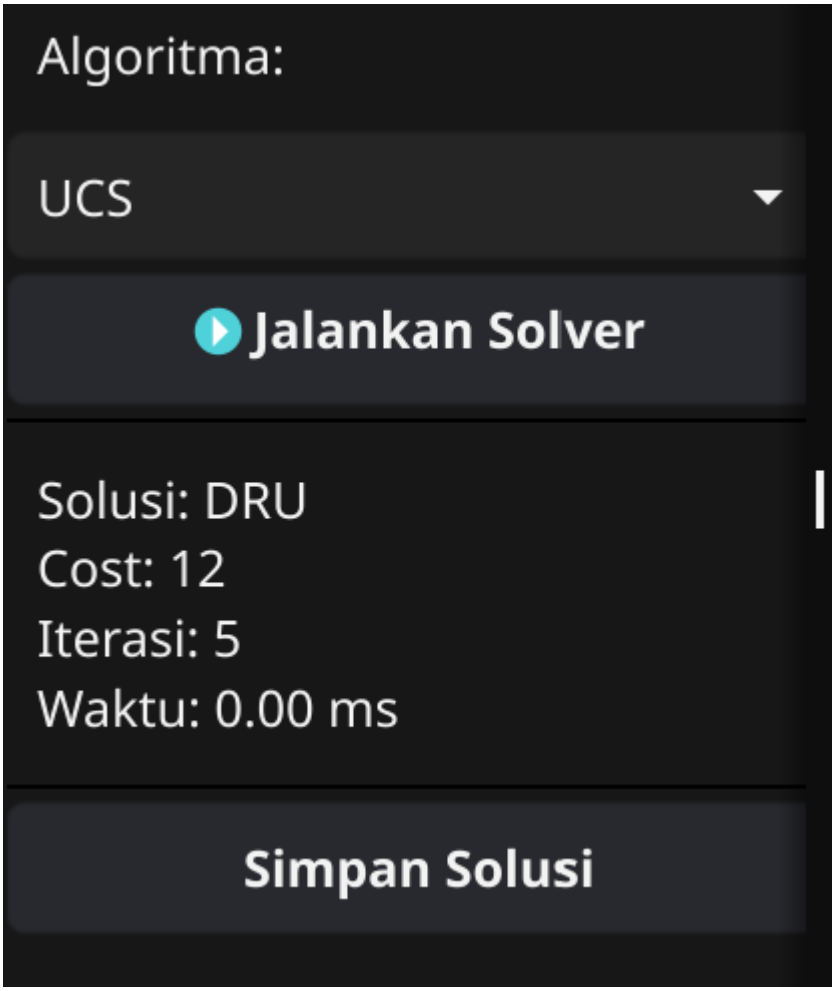
5.5 Contoh Test Case 5

Input



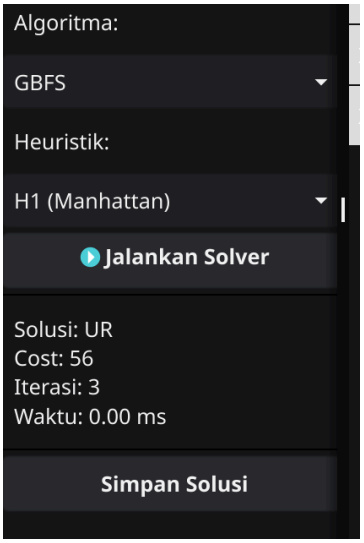
Gambar 5.5.1 input test case 5

Output — UCS

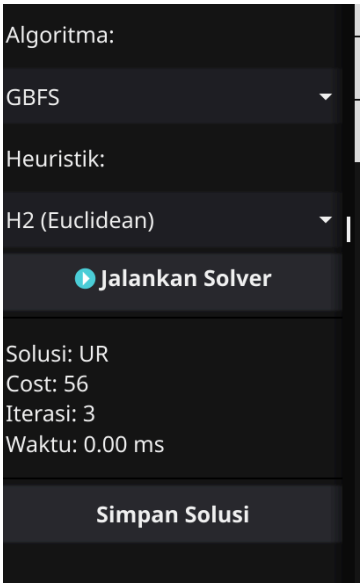


Gambar 5.5.2 Output UCS test case 5

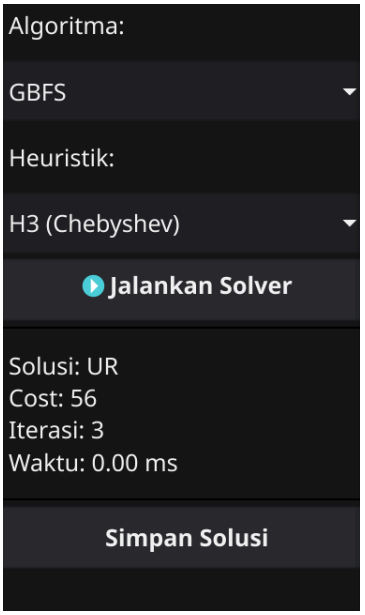
Output — GBFS



Gambar 5.5.3 Output GBFS H1(test case 5)

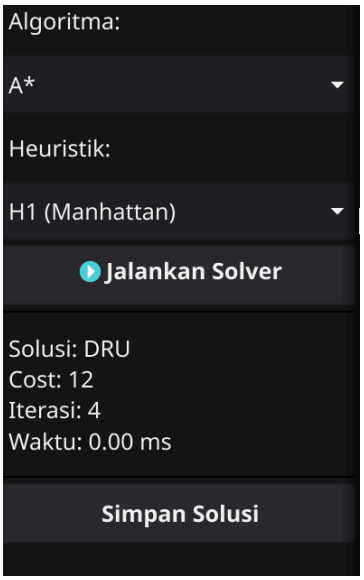


Gambar 5.5.4 Output GBFS H2(test case 5)

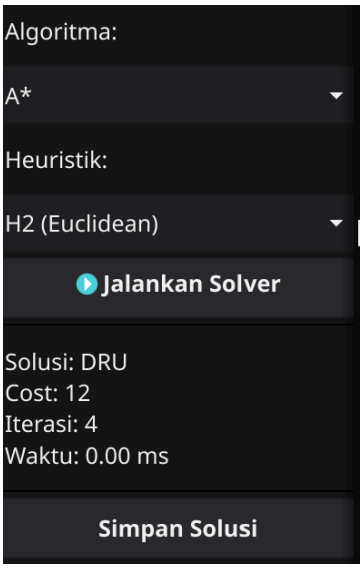


Gambar 5.5.5 Output GBFS H3(test case 5)

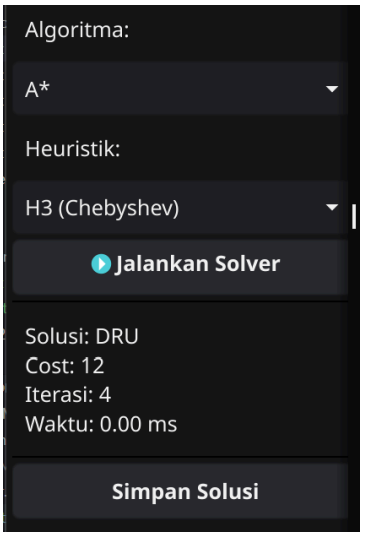
Output — A*



Gambar 5.5.6 Output A* H1(test case 5)



Gambar 5.5.7 Output A* H2(test case 5)



Gambar 5.5.8 Output A* H3(test case 5)

5.6 Analisis Hasil Percobaan

5.6.1 Perbandingan Cost Solusi

Tabel 5.6.1.1 perbandingan cost solusi

Test Case	UCS	GBFS H1	GBFS H2	GBFS H3	A* H1	A* H2	A* H3
1	Tidak ada solusi	Tidak ada solusi	Tidak ada solusi	Tidak ada solusi	Tidak ada solusi	Tidak ada solusi	Tidak ada solusi
2	Tidak ada solusi	Tidak ada solusi	Tidak ada solusi	Tidak ada solusi	Tidak ada solusi	Tidak ada solusi	Tidak ada solusi
3	12	56	56	56	12	12	12
4	15	15	15	15	15	15	15
5	12	56	56	12	12	12	12

Test case 1 tidak memiliki solusi karena struktur peta mengharuskan pin melewati checkpoint '1' yang terletak tepat di sebelah kiri tile Lava. setiap jalur menuju checkpoint tersebut mengharuskan pin bergerak melewati atau berhenti di tile Lava sehingga selalu game over. Test case 2 tidak memiliki solusi karena baris ketiga papan sepenuhnya terisi dinding yang memisahkan area start dan area goal secara total, sehingga tidak ada jalur yang dapat menghubungkan keduanya.

Pada test case yang memiliki solusi (TC 3, 4, 5), UCS dan A* (semua varian heuristik) selalu menghasilkan solusi dengan cost optimal. Pada test case 3 dan 5, GBFS menghasilkan solusi UR dengan cost 56, jauh lebih tinggi dibandingkan solusi optimal DRU dengan cost 12 yang ditemukan UCS dan A*. Hal ini terjadi karena GBFS hanya meminimalkan estimasi jarak geometris ke tujuan (h(n)) tanpa mempertimbangkan cost tile yang sebenarnya sangat mahal di baris atas (cost 9 per tile). Pada test case 4, semua algoritma sepakat pada solusi optimal RDRUD dengan cost 15 karena ruang pencarian yang terbatas hanya menyisakan satu jalur valid akibat constraint checkpoint berurutan.

5.6.2 Perbandingan Jumlah Iterasi

Tabel 5.6.2.1 perbandingan jumlah iterasi solusi

Test Case	UCS	GBFS H1	GBFS H2	GBFS H3	A* H1	A* H2	A* H3
1	15	20	25	25	15	15	15
2	5	5	5	5	5	5	5
3	5	3	3	3	4	4	4
4	10	7	6	6	9	9	9
5	5	3	3	3	4	4	4

Pada test case yang memiliki solusi, GBFS secara konsisten mengekspansi node paling sedikit dibandingkan A* dan UCS. Pada TC 3 dan TC 5, GBFS hanya membutuhkan 3 iterasi, A* 4 iterasi, dan UCS 5 iterasi. Ini karena GBFS langsung mengejar node yang paling dekat secara heuristik ke tujuan tanpa mempertimbangkan cost, sehingga cepat menemukan suatu solusi meski tidak optimal.

Pada test case tanpa solusi (TC 1), perilaku menarik terjadi: GBFS justru mengekspansi lebih banyak node (20–25 iterasi) dibanding UCS dan A* (15 iterasi). GBFS terus mengejar node dengan h(n) kecil namun tidak produktif, sedangkan UCS dan A* lebih sistematis dalam menguras ruang pencarian sehingga lebih cepat menyimpulkan tidak ada solusi.

UCS memiliki iterasi terbanyak di antara ketiga algoritma pada semua test case bersolusi karena ekspansinya sepenuhnya berbasis cost aktual g(n) tanpa panduan arah, sehingga banyak node yang diekspansi sebelum menemukan goal.

5.6.3 Kesimpulan Analisis

Berdasarkan hasil percobaan pada kelima test case, dapat ditarik beberapa kesimpulan sebagai berikut.

UCS terbukti selalu menemukan solusi optimal (cost minimum) pada setiap test case yang memiliki solusi, namun dengan jumlah iterasi terbanyak. Ini sesuai dengan sifat teoritis UCS sebagai algoritma complete dan optimal, tetapi tidak efisien karena tidak memiliki panduan heuristik.

A* dengan ketiga varian heuristik (Manhattan, Euclidean, Chebyshev) juga selalu menghasilkan solusi optimal dan dengan iterasi yang lebih sedikit dari UCS. Ketiga heuristik menghasilkan hasil yang identik pada semua test case, menunjukkan bahwa pada peta berskala kecil seperti yang diujikan, perbedaan karakteristik antar heuristik belum memberikan dampak yang signifikan. A* merupakan pilihan terbaik secara keseluruhan karena menggabungkan optimalitas solusi dengan efisiensi pencarian.

GBFS adalah algoritma paling efisien dalam hal jumlah iterasi, namun tidak menjamin solusi optimal. Pada TC 3 dan TC 5, GBFS menghasilkan solusi dengan cost 56 dibandingkan cost optimal 12 — selisih yang sangat besar. Ini merupakan trade-off fundamental GBFS: cepat menemukan solusi, tetapi kualitasnya tidak terjamin. GBFS cocok digunakan ketika kecepatan pencarian lebih diprioritaskan daripada kualitas solusi.

Bab VI — Source Code

6.1 Repository

https://github.com/FudhailFayyadh/Tucil3_18223057_18223121

Lampiran — Checklist Spesifikasi

No	Poin	Ya	Tidak
1	Program berhasil dikompilasi tanpa kesalahan		
2	Program berhasil dijalankan		
3	Solusi yang diberikan program benar dan mematuhi aturan permainan		
4	Program dapat membaca masukan berkas .txt serta menyimpan solusi dalam berkas .txt		
5	Program memiliki Graphical User Interface (GUI)		
6	Ada algoritma pathfinding alternatif selain dari spesifikasi wajib (Algoritma X dan Algoritma Y)		
7	Ada tambahkan dua alternatif heuristik selain yang utama (Heuristik X dan Heuristik Y)		

Pernyataan Tidak Melakukan Kecurangan

Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (Generative AI), melainkan hasil pemikiran dan analisis mandiri.

Stanislaus Ardy Bramantyo

Fudhail Fayyadh